SISTEMAS OPERATIVOS MONOPUESTO (SMR 1.º)

# Solucionario y criterios · Examen de los temas 02 a 05

**Tipo** Solucionario | **Nivel** 1 · Básico | **Tema** 02 a 05 | **CM** CM1, CM3, CM4

Documento del profesor. **No se publica.** Puntuación: A (test) 4 + B (cortas) 3 + C (ejercicios) 3 = 10. Las respuestas del test están repartidas: cinco a, cinco b, cinco c y cinco d.

## Parte A · Test (0,2 por acierto, los errores no restan)

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|
| **b** | **c** | **a** | **d** | **c** | **b** | **a** | **d** | **c** | **b** |

| 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 |
|---|---|---|---|---|---|---|---|---|---|
| **a** | **d** | **b** | **d** | **b** | **c** | **a** | **d** | **c** | **a** |

Por bloque: sistema operativo 1–3 · instalación y particiones 4–8 · procesos 9–12 · comandos 13–17 · árbol de carpetas 18–20.

> ℹ️ **Nota.** Distractores pensados a propósito: en la 16, `ren` es el comando de Windows (CMD); en la 19, «la raíz del sistema» es la confusión típica entre `/` y `/root`; en la 9, la opción b) es la respuesta correcta al revés.

## Parte B · Preguntas cortas (0,5 cada una)

**B1. Funciones del sistema operativo.** Gestionar procesos (CPU), gestionar la memoria (RAM), gestionar ficheros (disco), gestionar dispositivos (entrada/salida), usuarios y seguridad, e interfaz. Ejemplos válidos: música mientras navegas (procesos), cerrar un juego libera RAM (memoria), guardar un trabajo en una carpeta (ficheros), pinchar un USB (dispositivos), pedir la contraseña al entrar (seguridad), el escritorio o el terminal (interfaz).

- 0,25: las 6 funciones (5 → 0,2; 3 o 4 → 0,1).
- 0,25: dos ejemplos que encajen con su función (0,125 cada uno).

**B2. Máquina virtual y sesión *live*.** Una MV es un ordenador simulado por software (VirtualBox) dentro del ordenador real: sirve para instalar y probar sistemas sin tocar el equipo de verdad. Una sesión *live* es el sistema operativo funcionando desde la ISO (o un USB), sin instalarlo en el disco: sirve para probarlo antes de instalar y para rescatar un equipo que no arranca.

- 0,25 cada concepto (0,15 la definición + 0,1 para qué sirve).

**B3. Particiones.**

| Partición | Tamaño | Formato | Punto de montaje | Para qué sirve |
|---|---|---|---|---|
| 1 | 512 MB | FAT32 (partición del sistema EFI) | `/boot/efi` | Arrancar: la UEFI busca aquí el arranque |
| 2 | 15 GB | ext4 | `/` | El sistema y los programas |
| 3 | 2 GB | swap | — (no se monta) | Memoria de reserva cuando se llena la RAM |
| 4 | el resto | ext4 | `/home` | Los datos de los usuarios |

`/home` aparte: si se reinstala el sistema se formatea `/` sin tocar `/home`, y los datos se conservan.

- 0,1 por fila completa y correcta (0,05 si tiene un fallo) → 0,4.
- 0,1 la explicación de `/home`.

**B4. Estados de un proceso.** Nuevo, listo, en ejecución, bloqueado y terminado. Al pedir el fichero pasa a **bloqueado**; cuando llega el dato pasa a **listo** (no directamente a ejecución: espera a que el planificador le dé turno).

- 0,2: los 5 estados (4 → 0,1).
- 0,15: bloqueado. 0,15: listo. Si contesta «en ejecución» en el segundo paso, 0.

**B5. Rutas.**

```
cat /home/alumno/docs/a.txt    # absoluta
cat ../docs/a.txt              # relativa
```

La absoluta empieza por `/` (desde la raíz) y vale estés donde estés; la relativa depende de la carpeta en la que estás. También vale `cat ~/docs/a.txt` como forma corta de la absoluta (no como relativa).

- 0,15 cada comando. 0,2 la diferencia.

**B6. Carpetas.** `/etc` configuración del sistema · `/home` carpetas personales de los usuarios · `/dev` dispositivos (el disco es `/dev/sda`) · `/tmp` ficheros temporales, se vacía al reiniciar · `/var/log` registros

(logs) del sistema. `/` es la raíz de todo el árbol; `/root` es la carpeta personal del administrador. **No son lo mismo.**

- 0,08 por carpeta (0,4) + 0,1 la diferencia `/` y `/root`.

## Parte C · Ejercicios

### C1. Planificación (1,5)

Procesos: P1 = 6, P2 = 3, P3 = 1, P4 = 4. Todos llegan en 0. Espera = cuándo termina − lo que necesita.

**a) FCFS (0,4)** · orden P1, P2, P3, P4

```
| P1          | P2    |P3| P4      |
0             6       9  10        14
```

| Proceso | Termina | Espera |
|---|---|---|
| P1 | 6 | 0 |
| P2 | 9 | 6 |
| P3 | 10 | 9 |
| P4 | 14 | 10 |

Media = (0 + 6 + 9 + 10) / 4 = **6,25**

**b) SJF (0,4)** · orden P3, P2, P4, P1

```
|P3| P2    | P4      | P1          |
0  1       4         8             14
```

| Proceso | Termina | Espera |
|---|---|---|
| P3 | 1 | 0 |
| P2 | 4 | 1 |
| P4 | 8 | 4 |
| P1 | 14 | 8 |

Media = (0 + 1 + 4 + 8) / 4 = **3,25**

**c) Round Robin, cuanto 2 (0,5)**

| Turno | Proceso | De… a… | Le queda | Cola después |
|---|---|---|---|---|
| 1 | P1 | 0 → 2 | 4 | P2, P3, P4, P1 |
| 2 | P2 | 2 → 4 | 1 | P3, P4, P1, P2 |
| 3 | P3 | 4 → 5 | 0 ✓ | P4, P1, P2 |
| 4 | P4 | 5 → 7 | 2 | P1, P2, P4 |
| 5 | P1 | 7 → 9 | 2 | P2, P4, P1 |
| 6 | P2 | 9 → 10 | 0 ✓ | P4, P1 |
| 7 | P4 | 10 → 12 | 0 ✓ | P1 |
| 8 | P1 | 12 → 14 | 0 ✓ | — |

```
| P1 | P2 |P3| P4 | P1 |P2| P4 | P1 |
0    2    4  5    7    9  10   12   14
```

| Proceso | Termina | Espera |
|---|---|---|
| P1 | 14 | 14 − 6 = 8 |
| P2 | 10 | 10 − 3 = 7 |
| P3 | 5 | 5 − 1 = 4 |
| P4 | 12 | 12 − 4 = 8 |

Media = (8 + 7 + 4 + 8) / 4 = **6,75**

**d) (0,2)** La menor espera media es la de **SJF** (3,25). Windows y Linux usan variantes de Round Robin porque son sistemas **interactivos**: lo importante es que todos los programas reciban turno enseguida (el ratón y el teclado responden rápido) y que ninguno se quede esperando para siempre. Además, SJF necesita saber de antemano cuánto va a durar cada proceso, y eso no se sabe.

**Criterios C1.** En a), b) y c): la mitad por el Gantt bien ordenado y con los instantes, la otra mitad por las esperas y la media. Si arrastra un error de orden pero calcula bien las esperas de su propio diagrama, se le da la parte del cálculo. En c), el fallo típico es devolver al proceso al principio de la cola en lugar de al final: ese Gantt vale 0,1 como mucho.

## C2. Seguir los comandos (1,5)

**a) (0,5)** Al final:

```
/home/alumno/examen
├── d.txt          ← «tres» (era b.txt, se movió aquí con otro nombre)
├── fotos/         ← vacía (c.txt se borró en el comando 10)
└── textos/
    └── a.txt      ← «uno» y «dos»
```

- 0,15 `d.txt` en `examen` (no en `textos`). 0,15 `fotos` vacía. 0,1 `a.txt` en `textos`. 0,1 que no aparezcan `b.txt` ni `c.txt`.

**b) (0,25)** `/home/alumno/examen` (el comando 9 sube de `textos` a `examen`).

**c) (0,25)** Dos líneas: `uno` y `dos`.

**d) (0,25)** Sólo `dos`: un `>` borra lo que había antes.

**e) (0,25)** `rm -r ~/examen` o `rm -r /home/alumno/examen`. Desde donde está (dentro de `examen`), `rm -r ../examen` también funciona. Si contesta `cd ..` y después `rm -r examen` (dos comandos), 0,15. `rmdir examen` no vale porque la carpeta no está vacía: 0.

---